Lesson 13 — Feedback Control: PD and PID Fundamentals

Physical AI curriculum | Module 3: Robot Dynamics and Control | Lesson 13 of 40

Status: Completed (Python experiment passed; assessment answers B=A, C=B).

Learning objectives

- Describe closed-loop feedback for joint angle control.
- Explain proportional, derivative, and integral control.
- Build a PD controller with gravity compensation for a simplified one-joint arm.
- Simulate the joint dynamics and verify that it reaches its target.
- Recognize limitations of this educational model.

1. Intuition: from torque to controlled motion

Lesson 12 simulated motion under a prescribed motor torque. Lesson 13 computes motor torque continuously from the error between a target joint angle and the measured angle, together with joint angular velocity. This is closed-loop control.

Target angle → compute error → PD controller → motor torque → robot dynamics → measured angle and velocity → repeat.

2. Mathematics

Let `theta_target` be a stationary joint-angle target, `theta` the current angle, and `omega = d(theta)/dt`.

Position error: `e = theta_target - theta`

P control: `tau_P = Kp * e` — correction proportional to position error.

PD control: `tau_PD = Kp * e - Kd * omega` — additional velocity feedback damps motion, for stationary setpoints. In general the derivative term can be written `Kd * de/dt`.

PID control: `tau_PID = Kp * e + Ki * integral(e dt) + Kd * de/dt`. Integral action can counter steady disturbance but requires care regarding saturation and windup.

Single-joint dynamics (simplified):

`I * alpha + b * omega + m * g * L * cos(theta) = tau_motor`

`alpha = (tau_motor - b * omega - m*g*L*cos(theta)) / I`

Angle `theta` is measured counterclockwise from horizontal. The model treats the load as a point mass at distance `L`, with constant inertia `I`, viscous damping `b`, and no motor saturation.

PD plus gravity compensation:

`tau_motor = Kp*(theta_target-theta) - Kd*omega + m*g*L*cos(theta)`

For the exact model, substituting this into the dynamics gives `I*alpha = Kp*e - (Kd+b)*omega`; gravity cancels. This is idealized model-based compensation.

3. Complete Python experiment

Save as `lesson_13_pd_controller.py`. Requires `numpy` and `matplotlib`.

```
# Python example
import numpy as np
import matplotlib.pyplot as plt

# Robot parameters
I = 0.5
b = 0.2
m = 1.0
L = 0.5
g = 9.81

# Controller gains
Kp = 12.0
Kd = 3.0

# Target and initial state
target = np.deg2rad(45.0)
theta = 0.0
omega = 0.0

dt = 0.005
duration = 3.0
steps = int(duration / dt)

time_log = []
angle_log = []
torque_log = []

for step in range(steps):
    time = step * dt
    error = target - theta
    gravity_comp = m * g * L * np.cos(theta)
    motor_torque = Kp * error - Kd * omega + gravity_comp

    alpha = (motor_torque - b * omega
             - m * g * L * np.cos(theta)) / I

    # Semi-implicit Euler integration
    omega += alpha * dt

    theta += omega * dt

    time_log.append(time)
    angle_log.append(np.rad2deg(theta))
    torque_log.append(motor_torque)

final_angle = np.rad2deg(theta)
final_error = abs(45.0 - final_angle)
```

```
print("Target angle: 45.00 degrees")
print(f"Final angle: {final_angle:.3f} degrees")
print(f"Final error: {final_error:.4f} degrees")
print("Verification: PASSED" if final_error < 1.0
      else "Verification: FAILED")

plt.figure(figsize=(8, 5))
plt.plot(time_log, angle_log, label="Joint angle")
plt.axhline(45.0, linestyle="--", label="Target 45 deg")
plt.xlabel("Time (seconds)")
plt.ylabel("Joint angle (degrees)")
plt.title("Lesson 13: PD Joint Position Control")
plt.grid(True)
plt.legend()
plt.tight_layout()
plt.savefig("lesson_13_pd_response.png", dpi=160)
plt.show()
```

Run: `python lesson_13_pd_controller.py`

Acceptance test: Final angular error less than 1 degree, with output `Verification: PASSED`. The learner reported that the program passed and produced its plot; the exact printed final angle was not supplied.

4. Interpreting the response

- `Kp` determines the strength of the restoring action. Too little can make response slow; too much can lead to overshoot or instability in realistic systems.

- `Kd` resists angular velocity and generally improves damping. Excessive derivative gain may amplify measurement noise or make control unnecessarily aggressive.

- `Ki` accumulates error; it is not implemented in this Python experiment. Practical PID requires actuator limits and anti-windup provisions.

- Gravity compensation is based on an assumed model. Real robots have model mismatch, friction, sensor noise, motor limits, and latency.

5. Connection to Isaac Sim and Physical AI

Earlier lessons calculated desired joint angles by inverse kinematics. A feedback controller generates commands to drive a physical, dynamic joint toward those angles. The current two-link Isaac Sim demonstration uses visual transforms only; it does not yet simulate dynamic actuator torque. Future experiments will use articulated joints and their drives.

6. Assessment record

The user reported:

Component | Response | Result

Python PD control experiment | PASSED | Pass

Purpose of proportional term | A: corrective torque proportional to position error | Correct

Purpose of derivative feedback | B: damping and reduced overshoot | Correct

Lesson status: COMPLETE.

7. Review prompts

1. Why does a P controller increase its action when the target is far away?
2. Why does the derivative term often reduce oscillation?
3. Why might constant gravity compensation be insufficient for an articulated arm in varying poses?
4. What practical problem arises when the integral term keeps accumulating while a motor is saturated?

Next: Lesson 14 — Controller Tuning and Stability (accelerated).